2110573 PATTERN RECOGNITION

Homework 5: PyTorch & Transformer

Computer Engineering, Chulalongkorn Univerity

Worralop Srichainont

March 4, 2026

Contents

I Precipitation Nowcasting **1**

1 Problem T1 **2**

2 Problem T2 **2**

3 Problem T3 **3**

4 Problem T4 **4**

5 Problem T5 **5**

6 Problem T6 **5**

7 Problem T7 **6**

8 Problem T8 **6**

II Scheduler & Evaluation **7**

9 Problem T9 **7**
9.1 Implementation 7
9.2 Training 8

10 Problem T10 **10**

11 Problem T11 **11**

12 Problem T12 **12**

13 Problem T13 **14**
13.1 Plotting 14
13.2 Evaluation 15

III Convolutional Neural Networks **16**

14 Problem T14 **16**

15 Problem T15 **18**

16 Problem T16 **19**
16.1 Model Training 19
16.2 Plotting 20
16.3 Evaluation 21

IV Transformer **22**

17 Problem T17 **23**
17.1 Dataset Class 23
17.2 Create DataLoader 24

18 Problem T18 **25**

19 Problem T19 **26**
19.1 Self Attention Class 26
19.2 Transformer Model 27

20 Problem T20 **28**
20.1 Model Training 28
20.2 Evaluation 29

V PyTorch Playground **30**

21 Problem T21 **30**
21.1 Data Preparation 30
21.2 LSTM Model Class 30

22 Problem T22 **32**
22.1 Model Training 32
22.2 Evaluation 33

23 Problem T23 **34**
23.1 Test Dataset Evaluation 34
23.2 Explanation 34

Part I
Precipitation Nowcasting

Precipitation nowcasting is the the task of predicting the amount of rainfall in a certain region given some kind of sensor data. The term nowcasting refers to tasks that try to predict the current or near future conditions (within 6 hours).

You will be given satellite images in 3 different bands covering a 5 by 5 region from different parts of Thailand. In other words, your input will be a $5 \times 5 \times 3$ image. Your task is to predict the amount of rainfall in the center pixel. You will first do the prediction using just a simple fully-connected neural network that view each pixel as different input features.

Since the your input is basically an image, we will then view the input as an image and apply CNN to do the prediction. Finally, we can also add a time component since weather prediction can benefit greatly using previous time frames. Each data point actually contain 5 time steps, so each input data point has a size of $5 \times 5 \times 5 \times 3$ (time $\times$ height $\times$ width $\times$ channel), and the output data has a size of 5 (time). You will use this time information when you work with RNNs.

Finally, we would like to thank the Thai Meteorological Department for providing the data for this assignment.

Please access the Kaggle Notebook here: `https://www.kaggle.com/code/worralopsrichainont/2110573-patt-recog-hw5`

1 Problem T1

Explain each line of code in the function `preprocess_for_ff()`

Problem T1

```python
def preprocess_for_ff(x_train, y_train, x_val, y_val):
    x_train_ff = x_train.reshape((-1, 5 * 5 * 3))
    y_train_ff = y_train.reshape((-1, 1))
    x_val_ff = x_val.reshape((-1, 5 * 5 * 3))
    y_val_ff = y_val.reshape((-1, 1))
    x_test_ff = x_test.reshape((-1, 5 * 5 * 3))
    y_test_ff = y_test.reshape((-1, 1))

    return x_train_ff, y_train_ff, x_val_ff, y_val_ff, x_test_ff, y_test_ff
```

Answer

1. The original image are 5×5 pixels with RGB colors channel.
2. Flatten each $5 \times 5 \times 3$ image from `x_train` to a 1D array of 75 numbers.
3. Reshape the training labels (`y_train`) to a `sample_size` $\times 1$ 2D array.
4. Repeat the step 2-3 for validation dataset (`x_val` and `y_val`) and test dataset (`x_test` and `y_test`)
5. Return the preprocessed data

2 Problem T2

Why is the loss MSE?

Problem T2

```python
loss_fn = nn.MSELoss()
```

Ans. The model is trying to predict the amount of rainfall at a given time step and the rainfall is the continuous numerical value. Therefore, it is the regression task and MSE loss is suitable for it.

3 Problem T3

Below, the code for creating a 3-layers fully connected neural network in PyTorch is provided. Run the code and make sure you understand what you are doing. Then, report the results.

Problem T3

```python
class FeedForwardNN(nn.Module):
    def __init__(self, hidden_size=200):
        super(FeedForwardNN, self).__init__()
        self.ff1 = nn.Linear(75, hidden_size)
        self.ff2 = nn.Linear(hidden_size, hidden_size)
        self.ff3 = nn.Linear(hidden_size, hidden_size)
        self.out = nn.Linear(hidden_size, 1)

    def forward(self, x):
        hd1 = F.relu(self.ff1(x))
        hd2 = F.relu(self.ff2(hd1))
        y = F.relu(self.ff3(hd2))
        y = self.out(y)
        return y.reshape(-1, 1)
```

Answer

- The final dense layer has no activation function as can be seen in `y = self.out(y)`.
- **ReLU** would actually be a better activation function for the final dense layer, because the rainfall cannot be negative and ReLU can ensure that every prediction are not negative rainfall value.

4 Problem T4

Explain why the first linear layer has number of parameters equals to 15200?

Layer	Output Shape	Parameters
FeedForwardNN	[1024, 1]	-
Linear: 1-1	[1024, 200]	15,200
Linear: 1-2	[1024, 200]	40,200
Linear: 1-3	[1024, 200]	40,200
Linear: 1-4	[1024, 1]	201

Table 1: Model Summary

Answer

According to the following `FeedForwardNN` model code.

Problem T4

```python
class FeedForwardNN(nn.Module):
    def __init__(self, hidden_size=200):
        super(FeedForwardNN, self).__init__()
        self.ff1 = nn.Linear(75, hidden_size)
        # MORE LAYERS...
```

The code shows that the layer `ff1` has 75 input nodes and 200 output nodes

- **Weight**: $75 \times 200 = 15,000$ parameters
- **Bias**: 200 parameters
- **Total**: 15,000 + 200 = 15,200 parameters

5 Problem T5

Plot `loss` and `val_loss` as a function of epochs.

Answer

Use this following `matplotlib` code to plot the loss of each epoch.

Problem T5

```
# Generate a sequence of epoch numbers for the x-axis
epochs = range(1, len(train_losses) + 1)

# Create the plot
plt.figure(figsize=(10, 6))
plt.plot(epochs, train_losses, 'b-', label='Training Loss (MSE)')
plt.plot(epochs, val_losses, 'r-', label='Validation Loss (MSE)')

# Add labels, title, and legend
plt.title('Training and Validation Losses over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE)')
plt.legend()
plt.grid(True)

# Display the plot
plt.show()
```

The result shows as follows

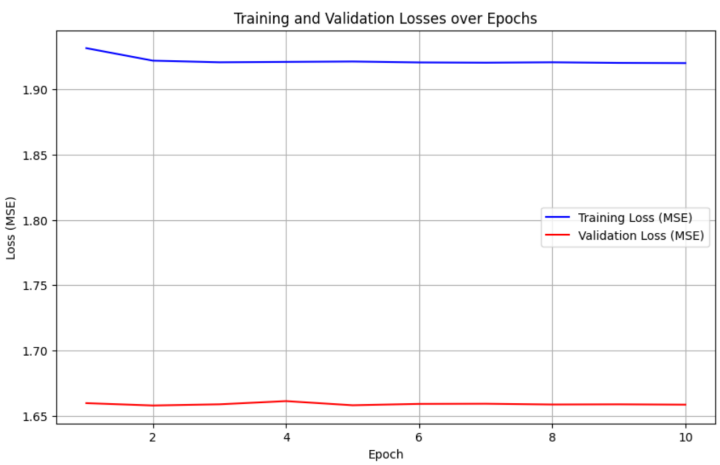


Figure 1: Fully Connected Model Loss

6 Problem T6

When does the model starts to overfit?

Ans. According to the graph in the problem T5, the model starts to overfit around epoch 2 or 3, because it starts to get better at predicting the training data starts getting worse at predicting validation data.

7 Problem T7

Plot the learning rate as a function of the epochs.

Answer

Use this following `matplotlib` code to plot the learning rate of each epoch.

Problem T7

```
# Generate a sequence of epoch numbers for the x-axis
epochs = range(1, len(learning_rates) + 1)

# Create the plot
plt.figure(figsize=(10, 6))
plt.plot(epochs, learning_rates, 'g-o', label='Learning Rate', linewidth=2, markersize=6)

# Add labels, title, and legend
plt.title('Learning Rate over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Learning Rate')
plt.legend()
plt.grid(True)

# Display the plot
plt.show()
```

The result shows as follows

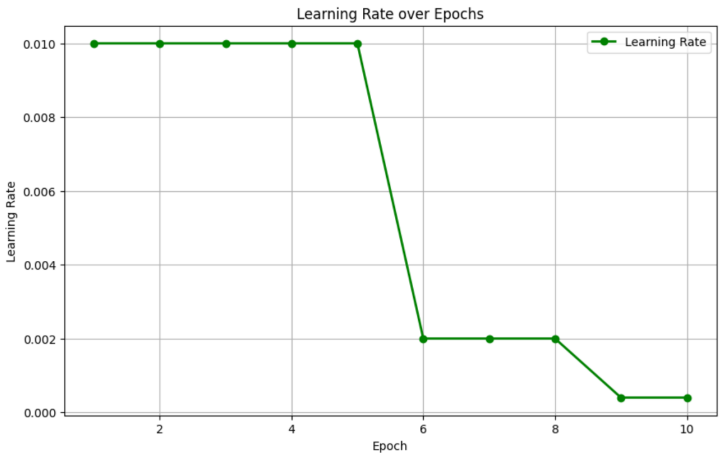


Figure 2: Fully Connected Model Learning Rate

8 Problem T8

What makes the learning rate change?

Ans. The learning rates changes because of the ReduceLROnPlateau scheduler. It monitors the validation loss and check if the validation loss stop getting lower for 2 epoch. If the model stop improving for 2 epochs, it reduces learning rate.

Part II
Scheduler & Evaluation

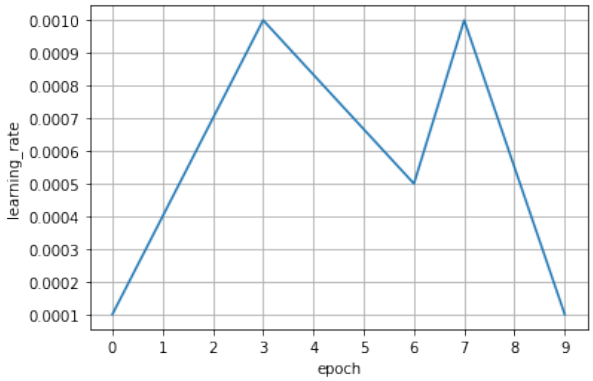
Use the code snippet below to load the model you just trained in the previous part

Load Model

```python
checkpoint = torch.load("model_ff.pth.tar")
loaded_model = FeedForwardNN(hidden_size=config["hidden_size"])  # Create model object
loaded_model.load_state_dict(checkpoint["model"])  # Load weights
print(f"Loaded epoch {checkpoint['epoch']} model")
```

9 Problem T9

The scheduler can be very complicated and you can write your own heuristic for it. Implement a custom learning rate scheduler that behaves like the following graph.



9.1 Implementation

Problem T9 - Implementation

```python
class MyScheduler:
    def __init__(self, optimizer: torch.optim.Optimizer):
        self.optimizer = optimizer
        self.base_lrs = [group['lr'] for group in optimizer.param_groups]

    def step(self, epoch):
        scale_factor = 0.8 ** epoch

        # Apply the new learning rate to the optimizer
        for i, group in enumerate(self.optimizer.param_groups):
            new_lr = self.base_lrs[i] * scale_factor
            group['lr'] = new_lr
```

9.2 Training

Problem T9 - Initialization

```python
# Initialize model and optimizer
model_ff = FeedForwardNN(hidden_size=config["hidden_size"]).to(device)
optimizer = torch.optim.Adam(model_ff.parameters(), lr=config["lr"])

# Initialize scheduler
my_scheduler = MyScheduler(optimizer)

# Track performance
train_losses = []
val_losses = []
learning_rates = []
```

Problem T9 - Training

```python
for epoch in range(config["epochs"]):
    # Training
    train_loss = []
    current_lr = optimizer.param_groups[0]["lr"]
    learning_rates.append(current_lr)

    # Flag model as training.
    model_ff.train()

    print(f"Training epoch {epoch+1}...")
    print(f"Current LR: {current_lr}")

    for i, (inputs, y_true) in enumerate(tqdm(train_loader)):
        # Transfer data from cpu to gpu
        inputs = inputs.to(device)
        y_true = y_true.to(device)

        # Reset the gradient
        optimizer.zero_grad()

        # Predict
        y_pred = model_ff(inputs)

        # Calculate loss
        loss = loss_fn(y_pred, y_true)

        # Compute gradient
        loss.backward()

        # Update parameters
        optimizer.step()

        # Log stuff
        train_loss.append(loss)

    avg_train_loss = torch.stack(train_loss).mean().item()
    train_losses.append(avg_train_loss)

    print(f"Epoch {epoch+1} train loss: {avg_train_loss:.4f}")

```

```python
    # Validation
    model_ff.eval()
    with torch.no_grad():  # No gradient is required during validation
        print(f"Validating epoch {epoch+1}")
        val_loss = []
        for i, (inputs, y_true) in enumerate(tqdm(val_loader)):
            # Transfer data from cpu to gpu
            inputs = inputs.to(device)
            y_true = y_true.to(device)

            # Predict
            y_pred = model_ff(inputs)

            # Calculate loss
            loss = loss_fn(y_pred, y_true)

            # Log stuff
            val_loss.append(loss)

        avg_val_loss = torch.stack(val_loss).mean().item()
        val_losses.append(avg_val_loss)
        print(f"Epoch {epoch+1} val loss: {avg_val_loss:.4f}")

    # Update the learning rate at the end of the epoch
    my_scheduler.step(epoch + 1)
```

10 Problem T10

Write a function to evaluate your model. Your function must predicts using the input model and return mean square error of the model.

Answer

Problem T10

```python
def evaluate(data_loader, model):
    """
    Evaluate model on validation data given by data_loader
    """
    # Set the model to evaluation mode
    model.eval()

    # Initialize MSE loss function
    loss_fn = nn.MSELoss()

    # Safely get the device the model is currently on
    device = next(model.parameters()).device

    batch_losses = []

    # Disable gradient calculation to save memory and speed up inference
    with torch.no_grad():
        for inputs, y_true in data_loader:
            # Transfer data to the correct device (GPU/CPU)
            inputs = inputs.to(device)
            y_true = y_true.to(device)

            # Forward pass: make predictions
            y_pred = model(inputs)

            # Calculate MSE loss for this batch
            loss = loss_fn(y_pred, y_true)
            batch_losses.append(loss)

    # Calculate the mean of all batch losses and convert to a standard Python float
    mse = torch.stack(batch_losses).mean().item()

    return mse
```

The evaluation compared to the baseline (majority rule) are this follows:

Source	Train MSE	Validation MSE
Majority Baseline	1.944	1.675
Fully Connected Model	1.918	1.656

Table 2: Fully Connected Model Evaluation

11 Problem T11

You might notice that the 3-layered feedforward does not use dropout at all. Now, try adding dropout (dropout rate of 20%) to the model, run, and report the result again.

In this problem, write a feedforward model with dropout.

Answer

Problem T11

```python
class FeedForwardNNDropout(nn.Module):
    def __init__(self, hidden_size=200, dropout_rate=0.5):
        super(FeedForwardNNDropout, self).__init__()
        # Linear layers
        self.ff1 = nn.Linear(75, hidden_size)
        self.ff2 = nn.Linear(hidden_size, hidden_size)
        self.ff3 = nn.Linear(hidden_size, hidden_size)
        self.out = nn.Linear(hidden_size, 1)

        # Dropout layer
        # p is the probability of an element to be zeroed out
        self.dropout = nn.Dropout(p=dropout_rate)

    def forward(self, x):
        # Apply dropout after the ReLU activation for each hidden layer
        hd1 = self.dropout(F.relu(self.ff1(x)))
        hd2 = self.dropout(F.relu(self.ff2(hd1)))
        y = self.dropout(F.relu(self.ff3(hd2)))

        # Output layer (no dropout applied here)
        y = self.out(y)
        return y.reshape(-1, 1)
```

12 Problem T12

Please complete the code to train your dropout model.

Answer

Problem T12 - Initialization

```python
# Initialize the new dropout model
model_ff_dropout = FeedForwardNNDropout(hidden_size=config["hidden_size"], dropout_rate=0.5)
model_ff_dropout = model_ff_dropout.to(device)

# Initialize the optimizer and scheduler
optimizer = torch.optim.Adam(model_ff_dropout.parameters(), lr=config["lr"])
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, "min",
    factor=config["scheduler_factor"],
    patience=config["scheduler_patience"],
    min_lr=config["scheduler_min_lr"]
)

# Lists to keep track of losses
train_losses_dropout = []
val_losses_dropout = []
```

See training codeblock in the next page →

Problem T12 - Training

```python
for epoch in range(config["epochs"]):
    # TRAINING
    model_ff_dropout.train()
    train_loss = []

    current_lr = optimizer.param_groups[0]["lr"]
    print(f"\nTraining epoch {epoch+1} (LR: {current_lr})...")

    for inputs, y_true in tqdm(train_loader):
        inputs = inputs.to(device)
        y_true = y_true.to(device)

        optimizer.zero_grad()
        y_pred = model_ff_dropout(inputs)
        loss = loss_fn(y_pred, y_true)
        loss.backward()
        optimizer.step()

        train_loss.append(loss.detach())

    avg_train_loss = torch.stack(train_loss).mean().item()
    train_losses_dropout.append(avg_train_loss)

    # VALIDATION
    model_ff_dropout.eval()
    val_loss = []

    with torch.no_grad():
        for inputs, y_true in tqdm(val_loader):
            inputs = inputs.to(device)
            y_true = y_true.to(device)

            y_pred = model_ff_dropout(inputs)
            loss = loss_fn(y_pred, y_true)
            val_loss.append(loss.detach())

    avg_val_loss = torch.stack(val_loss).mean().item()
    val_losses_dropout.append(avg_val_loss)

    print(f"Epoch {epoch+1} | Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f}")

    # Step the scheduler based on the validation loss
    scheduler.step(avg_val_loss)
```

13 Problem T13

Plot the losses and MSE of the training and validation as before. Evaluate the dropout model's performance

13.1 Plotting

Use this following `matplotlib` code to plot the loss of each epoch.

Problem T13 - Plotting

```python
# Plot here
# Generate the x-axis values based on the number of epochs recorded
epochs_dropout = range(1, len(train_losses_dropout) + 1)

# Create the plot
plt.figure(figsize=(10, 6))
plt.plot(epochs_dropout, train_losses_dropout, 'b-', label='Training Loss (Dropout)')
plt.plot(epochs_dropout, val_losses_dropout, 'r-', label='Validation Loss (Dropout)')

# Add labels, title, and legend
plt.title('Training vs Validation Loss with Dropout')
plt.xlabel('Epoch')
plt.ylabel('Loss (MSE)')
plt.legend()
plt.grid(True)

# Display the plot
plt.show()
```

The result shows as follows

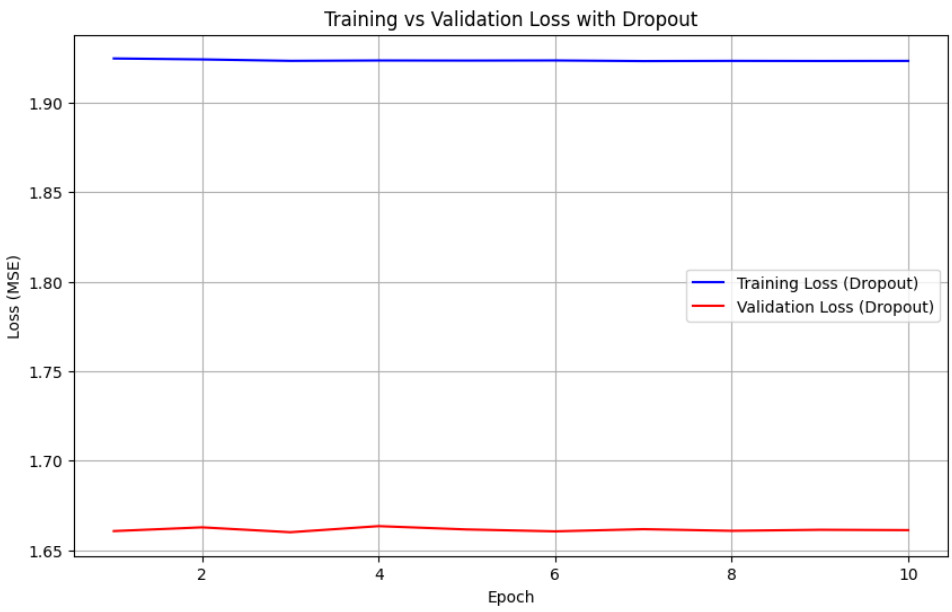


Figure 3: Fully Connected with Dropout Model Loss

13.2 Evaluation

Problem T13 - Evaluation

```
passdropout_mse = evaluate(val_loader, model_ff_dropout)
print(f"Dropout Model Validation MSE: {dropout_mse:.4f}")
```

The evaluation compared to the baseline (majority rule) are this follows:

Source	Train MSE	Validation MSE
Majority Baseline	1.944	1.675
Fully Connected Model	1.918	1.656
Fully Connected Model (Dropout)	1.923	1.661

Table 3: Fully Connected Model with Dropout Evaluation

Part III
Convolutional Neural Networks

Now let's try to incorporate the grid structure to your model. Instead of passing in vectors, we are going to pass in the 5×5 grid into the model. You are going to implement you own 2D Convolutional Neural Networks (CNN) with the following structure.

Layer	Output Shape	Parameters
Conv2DNN	-	-
Conv2d: 1-1	[1024, 200, 3, 3]	5,600
Linear: 1-2	[1024, 200]	360,200
Linear: 1-3	[1024, 200]	40,200
Linear: 1-4	[1024, 1]	201

Table 4: CNN Model Summary

14 Problem T14

Please complete the code for preparing data for training CNN.

Answer

Problem T14 - Functions

```python
def get_tensor_x_data_cnn(x_data):
    x_data_cnn = x_data[:, -1, :, :, :]
    tensor_x_data_cnn = torch.tensor(x_data_cnn, dtype=torch.float32)
    tensor_x_data_cnn = tensor_x_data_cnn.permute(0, 3, 1, 2)

    return tensor_x_data_cnn

def get_tensor_y_data_cnn(y_data):
    tensor_y_data_cnn = torch.tensor(y_data, dtype=torch.float32)
    if len(tensor_y_data_cnn.shape) > 1 and tensor_y_data_cnn.shape[1] > 1:
        tensor_y_data_cnn = tensor_y_data_cnn[:, -1].unsqueeze(1)

    return tensor_y_data_cnn
```

See the `DataLoader` code block in the next page →

Problem T14 - DataLoader

```python
# Arrange data
tensor_x_train_cnn = get_tensor_x_data_cnn(x_train)
tensor_y_train = get_tensor_y_data_cnn(y_train)

tensor_x_val_cnn = get_tensor_x_data_cnn(x_val)
tensor_y_val = get_tensor_y_data_cnn(y_val)

tensor_x_test_cnn = get_tensor_x_data_cnn(x_test)
tensor_y_test = get_tensor_y_data_cnn(y_val)

# Create PyTorch Datasets
train_dataset_cnn = TensorDataset(tensor_x_train_cnn, tensor_y_train)
val_dataset_cnn = TensorDataset(tensor_x_val_cnn, tensor_y_val)
test_dataset_cnn = TensorDataset(tensor_x_test_cnn, tensor_y_test)

# Create PyTorch DataLoaders
batch_size = 64

train_loader_cnn = DataLoader(train_dataset_cnn, batch_size=batch_size, shuffle=True)
val_loader_cnn = DataLoader(val_dataset_cnn, batch_size=batch_size, shuffle=False)
test_loader_cnn = DataLoader(test_dataset_cnn, batch_size=batch_size, shuffle=False)
```

15 Problem T15

Write a PyTorch Convolutional Neural Network (CNN) model.

Answer

Problem T15

```python
class CNNModel(nn.Module):
    def __init__(self):
        super(CNNModel, self).__init__()

        self.conv1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        self.fc1 = nn.Linear(32 * 1 * 1, 128)
        self.fc2 = nn.Linear(128, 1)

    def forward(self, x):
        # Convolutional layers with ReLU and Pooling
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))

        # Flatten the feature maps into a 1D vector
        x = torch.flatten(x, 1)

        # Fully connected layers
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

16 Problem T16

Please complete the code to train your CNN model

16.1 Model Training

Problem T16 - Initialization

```python
# Initialize the model
model_cnn = CNNModel()
model_cnn.to(device)

# Define Loss function and Optimizer
criterion = nn.MSELoss()
optimizer = optim.Adam(model_cnn.parameters(), lr=0.001)

# Training Loop
num_epochs = 10

# Storing losses
train_losses = []
val_losses = []
```

Problem T16 - Training

```python
for epoch in range(num_epochs):
    model_cnn.train() # Set model to training mode
    running_loss = 0.0

    # Store losses for the current batch
    batch_train_losses = []

    for i, data in enumerate(train_loader_cnn, 0):
        inputs, labels = data

        # Move data to GPU if available
        inputs, labels = inputs.to(device), labels.to(device)

        # Zero the parameter gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model_cnn(inputs)

        # Extract only the target for the last timestep
        if len(labels.shape) > 1 and labels.shape[1] > 1:
            labels = labels[:, -1].unsqueeze(1)

        # Ensure labels have the same shape as outputs for MSELoss
        labels = labels.view_as(outputs)

        loss = criterion(outputs, labels)

        # Backward pass and optimize
        loss.backward()
        optimizer.step()
```

```python

        batch_train_losses.append(loss.item())

    # Average loss for the epoch
    epoch_train_loss = sum(batch_train_losses) / len(batch_train_losses)
    train_losses.append(epoch_train_loss)

    # Evaluate on validation set after each epoch
    epoch_val_loss = evaluate(val_loader_cnn, model_cnn)
    val_losses.append(epoch_val_loss)

    print(f"Epoch {epoch+1}/{num_epochs} - Train MSE: {epoch_train_loss:.4f}, Val MSE: {epoch_val_loss:.4f}")
```

16.2 Plotting

Problem T16 - Plotting

```python
plt.figure(figsize=(10, 5))
plt.plot(train_losses, label='Training Loss')
plt.plot(val_losses, label='Validation Loss')
plt.title('CNN Model Loss (MSE) Over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Mean Squared Error')
plt.legend()
plt.grid(True)
plt.show()
```

The result shows as follows

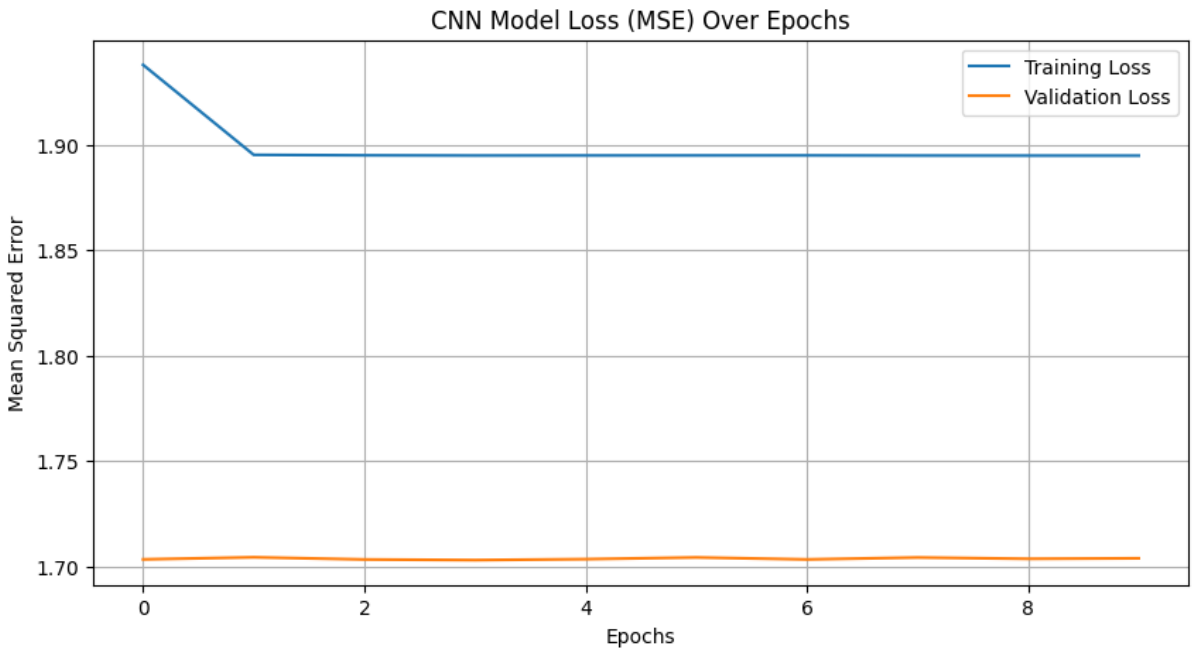


Figure 4: CNN Model Loss

16.3 Evaluation

Problem T16 - Evaluation

```
train_mse = evaluate(train_loader_cnn, model_cnn)
val_mse = evaluate(val_loader_cnn, model_cnn)

print(f"Final Training MSE: {train_mse:.4f}")
print(f"Final Validation MSE: {val_mse:.4f}")
```

The evaluation compared to the baseline (majority rule) are this follows:

Source	Train MSE	Validation MSE
Majority Baseline	1.944	1.675
Fully Connected Model	1.918	1.656
Fully Connected Model (Dropout)	1.923	1.661
CNN Model	1.895	1.704

Table 5: CNN Model Evaluation

Part IV
Transformer

The above models are not usually used in practice due to its limited capability. Transformers are generally used by computer vision, natural language processing, and speech processing.

Layer	**Output Shape**	**Parameters**
TransformerModel	[1024, 1]	-
PositionalEncoding: 1-1	[1024, 5, 76]	-
Dropout: 2-1	[1024, 5, 76]	-
SelfAttention: 1-2	[1024, 5, 76]	-
Linear: 2-2	[1024, 5, 76]	5,852
Linear: 2-3	[1024, 5, 76]	5,852
Linear: 2-4	[1024, 5, 76]	5,852
Softmax: 2-5	[1024, 5, 5]	-
LayerNorm: 1-3	[1024, 5, 76]	760
Linear: 1-4	[1024, 5, 76]	5,582
LayerNorm: 1-5	[1024, 5, 76]	760
Linear: 1-6	[1024, 1]	381

Table 6: Transformer Model Summary

17 Problem T17

In our dataloader, we will add the output of this timestep (the number of precipitation) as an auxiliary input to predict the next timestep.

- **Input size**: [`batch_size`, 5, 76]
- **Output size**: [textttbatch_size, 1]

Additionally, we will mask the input at the dataloader to the attention from observing future values. Suppose that we want to predict timestep 3, we will mask the timestep 3, 4 and 5 in our input by setting it to zeros, and we will predict the timestep 3.

Please complete the code for preparing data for training Transformer

17.1 Dataset Class

Problem T17 - Dataset Class

```python
class TransformerDataset(Dataset):
    def __init__(self, X, y):
        # Convert to PyTorch tensors
        self.X = torch.tensor(X, dtype=torch.float32) if not isinstance(X, torch.Tensor) else X.clone().detach()
        self.y = torch.tensor(y, dtype=torch.float32) if not isinstance(y, torch.Tensor) else y.clone().detach()

        self.num_samples = self.X.shape[0]
        self.seq_len = self.X.shape[1]

    def __len__(self):
        return self.num_samples * self.seq_len

    def __getitem__(self, idx):
        # Determine which original sample and which timestep to predict
        sample_idx = idx // self.seq_len
        t = idx % self.seq_len

        # Extract the sequence and ensure targets are 1D
        X_seq = self.X[sample_idx].clone() # Shape: (5, 75)
        y_seq = self.y[sample_idx].clone().view(-1) # Shape: (5,)

        # Add the output of the previous timestep as an auxiliary input
        # Shift targets by 1
        y_shifted = torch.zeros(self.seq_len, 1)
        y_shifted[1:] = y_seq[:-1].unsqueeze(1)

        # Concatenate features and auxiliary input to get (5, 76)
        X_combined = torch.cat((X_seq, y_shifted), dim=1)

        # Mask current and future timesteps
        # Suppose we want to predict t = 2, we mask t = 2,3,4 to zeros.
        X_combined[t:, :] = 0.0

        # The target is the actual precipitation at timestep t
        target = y_seq[t].unsqueeze(0) # Shape: (1,)

        return X_combined, target
```

17.2 Create DataLoader

Problem T17 - DataLoader

```python
# Reshape the input data to 3 dimensions: (entries, time-step, features)
# Flattening the spatial and channel dimensions into a single feature dimension
x_train_trans = x_train.reshape(x_train.shape[0], x_train.shape[1], -1)
x_val_trans = x_val.reshape(x_val.shape[0], x_val.shape[1], -1)
x_test_trans = x_test.reshape(x_test.shape[0], x_test.shape[1], -1)

# Create the custom datasets using your exact 'y' variable names
train_dataset_trans = TransformerDataset(x_train_trans, y_train)
val_dataset_trans = TransformerDataset(x_val_trans, y_val)
test_dataset_trans = TransformerDataset(x_test_trans, y_test)

# Create DataLoaders
batch_size = 1024

train_loader_trans = DataLoader(train_dataset_trans, batch_size=batch_size, shuffle=True)
val_loader_trans = DataLoader(val_dataset_trans, batch_size=batch_size, shuffle=False)
test_loader_trans = DataLoader(test_dataset_trans, batch_size=batch_size, shuffle=False)
```

18 Problem T18

Please write a PyTorch `PositionalEncoding` Layer.

Problem T18

```python
class PositionalEncoding(nn.Module):
    def __init__(self, seq_len, emb_dim, dropout=0.2):
        super(PositionalEncoding, self).__init__()
        self.dropout = nn.Dropout(p=dropout)

        # Create a matrix of shape (seq_len, emb_dim) to hold positional encodings
        pe = torch.zeros(seq_len, emb_dim)

        # Create a vector of positions (0, 1, 2, ..., seq_len-1)
        position = torch.arange(0, seq_len, dtype=torch.float).unsqueeze(1)

        # Calculate the division term for the sine and cosine arguments
        div_term = torch.exp(torch.arange(0, emb_dim, 2).float() * (-math.log(10000.0) / emb_dim))

        # Apply sine to even indices
        pe[:, 0::2] = torch.sin(position * div_term)

        # Apply cosine to odd indices
        pe[:, 1::2] = torch.cos(position * div_term)

        # Add a batch dimension: (1, seq_len, emb_dim)
        pe = pe.unsqueeze(0)

        # Register pe as a buffer (it's not a trainable parameter, but stays with the model)
        self.register_buffer('pe', pe)

    def forward(self, x):
        """
        Args:
            x: Input tensor of shape (batch_size, seq_len, emb_dim)
        """
        # Add the positional encoding to the input embedding
        x = x + self.pe[:, :x.size(1), :]
        return self.dropout(x)
```

19 Problem T19

Please write a PyTorch Transformer model.

19.1 Self Attention Class

Problem T19 - Self Attention Class

```python
class SelfAttention(nn.Module):
    def __init__(self, input_dim):
        super(SelfAttention, self).__init__()
        # Query, Key, and Value linear transformations
        self.q = nn.Linear(input_dim, input_dim)
        self.k = nn.Linear(input_dim, input_dim)
        self.v = nn.Linear(input_dim, input_dim)
        self.softmax = nn.Softmax(dim=-1)

    def forward(self, x):
        # x shape: (batch_size, seq_len, input_dim)
        Q = self.q(x)
        K = self.k(x)
        V = self.v(x)

        # Attention scores
        attention_scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(x.size(-1))
        attention_weights = self.softmax(attention_scores)

        # Weighted sum
        out = torch.matmul(attention_weights, V)
        return out
```

19.2 Transformer Model

Problem T19 - Transformer Model

```python
class TransformerModel(nn.Module):
    def __init__(self, seq_len=5, input_dim=76):
        super(TransformerModel, self).__init__()

        # Positional Encoding
        self.pos_encoder = PositionalEncoding(seq_len, input_dim)

        # Self Attention
        self.attention = SelfAttention(input_dim)

        # LayerNorm 1: Normalizing over (seq_len, input_dim) -> 760 params
        self.norm1 = nn.LayerNorm((seq_len, input_dim))

        # Feed-Forward Linear layer -> 5852 params
        self.linear_ffn = nn.Linear(input_dim, input_dim)

        # LayerNorm 2: -> 760 params
        self.norm2 = nn.LayerNorm((seq_len, input_dim))

        # Final Output layer -> 381 params
        # It takes the flattened output (5 * 76 = 380) and predicts 1 value
        self.fc_out = nn.Linear(seq_len * input_dim, 1)

    def forward(self, x):
        # Apply positional encoding
        x = self.pos_encoder(x)

        # Self-Attention with a residual connection
        attn_out = self.attention(x)
        x = self.norm1(x + attn_out)

        # Feed-Forward layer with a residual connection
        ffn_out = self.linear_ffn(x)
        x = self.norm2(x + ffn_out)

        # Flatten the sequence and feature dimensions: (batch_size, 5 * 76)
        x = torch.flatten(x, start_dim=1)

        # Final prediction: (batch_size, 1)
        out = self.fc_out(x)
        return out
```

20 Problem T20

Complete the code to train your Transformer model

20.1 Model Training

Problem T20 - Initialization

```python
# Initialize model
model_transformer = TransformerModel(seq_len=5, input_dim=76)
model_transformer = model_transformer.to(device)

# Loss function
criterion = nn.MSELoss()

# Transformers optimizaer
optimizer = optim.Adam(model_transformer.parameters(), lr=0.0001)

# Number of epochs
num_epochs = 10

# Storing losses
train_losses = []
val_losses = []
```

Problem T20 - Training

```python
for epoch in range(num_epochs):
    model_transformer.train()
    epoch_train_loss = 0.0
    num_batches = 0

    for inputs, targets in train_loader_trans:
        inputs, targets = inputs.to(device), targets.to(device)

        optimizer.zero_grad()
        outputs = model_transformer(inputs)

        # Ensure target shape matches output shape
        targets = targets.view_as(outputs)

        loss = criterion(outputs, targets)
        loss.backward()
        optimizer.step()

        epoch_train_loss += loss.item()
        num_batches += 1

    # Calculate average training loss for the epoch
    avg_train_loss = epoch_train_loss / num_batches
    train_losses.append(avg_train_loss)

    # Evaluate on the validation set at the end of the epoch
    avg_val_loss = evaluate(val_loader_trans, model_transformer)
    val_losses.append(avg_val_loss)

```

```
    print(f"Epoch [{epoch+1}/{num_epochs}] | Train Loss: {avg_train_loss:.4f} | Val Loss: {avg_val_loss:.4f}")
```

20.2 Evaluation

Problem T20 - Training

```
final_train_mse = evaluate(train_loader_trans, model_transformer)
final_val_mse = evaluate(val_loader_trans, model_transformer)

print(f"Final Training MSE: {final_train_mse:.4f}")
print(f"Final Validation MSE: {final_val_mse:.4f}")
```

The evaluation compared to the baseline (majority rule) are this follows:

Source	Train MSE	Validation MSE
Majority Baseline	1.944	1.675
Fully Connected Model	1.918	1.656
Fully Connected Model (Dropout)	1.923	1.661
CNN Model	1.895	1.704
Transformer Model	1.920	1.658

Table 7: CNN Model Evaluation

Part V
PyTorch Playground

Now, train the best model you can do for this task. You can use any model structure and function available.

Remember that training time increases with the complexity of the model. You might find printing computation graphs helpful in debugging complicated models. Your model should be better than your CNN or Transformer model in the previous sections.

21 Problem T21

Please write a function that returns your best PyTorch model.

21.1 Data Preparation

Problem T21 - Data Preparation

```
train_loader_best = train_loader_trans
val_loader_best = val_loader_trans
```

21.2 LSTM Model Class

Problem T21 - LSTM Model Class

```
class BiLSTM_NowcastingModel(nn.Module):
    def __init__(self, input_dim=76, hidden_dim=128, num_layers=2, dropout=0.3):
        super(BiLSTM_NowcastingModel, self).__init__()

        # Bidirectional LSTM to capture temporal dependencies from past to future and vice versa
        self.lstm = nn.LSTM(
            input_size=input_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=True,
            dropout=dropout if num_layers > 1 else 0
        )

        # Fully connected layers for the final prediction
        # hidden_dim * 2 because it's bidirectional
        self.fc1 = nn.Linear(hidden_dim * 2, 64)
        self.layer_norm = nn.LayerNorm(64)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(dropout)

        # Output layer predicting the 1 value (precipitation)
        self.fc2 = nn.Linear(64, 1)

    def forward(self, x):
```

```python
        # x shape: [batch_size, seq_len, input_dim]

        # lstm_out shape: [batch_size, seq_len, hidden_dim * 2]
        lstm_out, (hidden_state, cell_state) = self.lstm(x)

        # Extract the output from the last timestep to predict the next timestep
        last_timestep_out = lstm_out[:, -1, :]

        # Pass through dense layers
        x = self.fc1(last_timestep_out)
        x = self.layer_norm(x)
        x = self.relu(x)
        x = self.dropout(x)

        out = self.fc2(x)
        return out

def get_best_model():
    """
    Returns an instance of the best model.
    Make sure the input_dim matches your dataloader (e.g., 76 based on previous steps).
    """
    # Initialize with the dimensions derived from your previous dataloaders
    model = BiLSTM_NowcastingModel(input_dim=76, hidden_dim=128, num_layers=2)
    return model
```

22 Problem T22

Please complete the code to train your best model

22.1 Model Training

Problem T22 - Initialization

```python
# Initialize model
best_model = get_best_model()
best_model = get_best_model().to(device)

# Loss function
criterion = nn.MSELoss()

# Optimizer
optimizer = optim.Adam(best_model.parameters(), lr=0.001)

# Epoch amount
num_epochs = 15

# Storing losses
best_train_losses = []
best_val_losses = []

# Storing best model weights
best_val_mse = float('inf')
best_model_weights = copy.deepcopy(best_model.state_dict())
```

Problem T22 - Training

```python
# Training Loop
for epoch in range(num_epochs):
    best_model.train() # Set to training mode
    epoch_train_loss = 0.0

    for inputs, targets in train_loader_best:
        # Move data to GPU/CPU
        inputs = inputs.to(device)
        targets = targets.to(device)

        # Zero the parameter gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = best_model(inputs)

        # Ensure target shape matches output shape exactly to avoid broadcasting issues
        targets = targets.view_as(outputs)

        # Calculate loss
        loss = criterion(outputs, targets)

        # Backward pass (compute gradients)
        loss.backward()
```

```python

        # Update weights
        optimizer.step()

        epoch_train_loss += loss.item()

    # Calculate average losses for this epoch
    avg_train_loss = epoch_train_loss / len(train_loader_best)
    best_train_losses.append(avg_train_loss)

    # Evaluate on the validation set
    avg_val_loss = evaluate(val_loader_best, best_model)
    best_val_losses.append(avg_val_loss)

    print(f"Epoch [{epoch + 1}/{num_epochs}] | Train MSE: {avg_train_loss:.4f} | Val MSE: {avg_val_loss:.4f}")

    # Save the best model weights based on validation loss
    if avg_val_loss < best_val_mse:
        best_val_mse = avg_val_loss
        best_model_weights = copy.deepcopy(best_model.state_dict())
```

22.2 Evaluation

Problem T22 - Evaluation

```python
# Evaluate best model on validation and test set
best_model.load_state_dict(best_model_weights)

val_mse = evaluate(val_loader_trans, best_model)
test_mse = evaluate(test_loader_trans, best_model)

print(f"Best Model - Validation MSE: {val_mse:.4f}")
print(f"Best Model - Test MSE: {test_mse:.4f}")
```

The evaluation compared to the baseline (majority rule) are this follows:

Source	Train MSE	Validation MSE
Majority Baseline	1.944	1.675
Fully Connected Model	1.918	1.656
Fully Connected Model (Dropout)	1.923	1.661
CNN Model	1.895	1.704
Transformer Model	1.920	1.658
LSTM Model	1.924	1.661

Table 8: LSTM Model Evaluation

23 Problem T23

Explain what helped and what did not help here.

23.1 Test Dataset Evaluation

Source	Test MSE
Majority Baseline	1.687
Fully Connected Model	1.164
Fully Connected Model (Dropout)	1.160
CNN Model	1.161
Transformer Model	1.155
LSTM Model	1.165

Table 9: Test Dataset Evaluation

23.2 Explanation

Based on the Test MSE results, all of the trained models performed much better than the simple Majority Baseline (1.687).

What Helped:

- **Transformer Model:** This model was the most successful. Its attention mechanism is excellent at understanding both the weather images and the time sequence.
- **Dropout:** Adding dropout to the Fully Connected Model slightly improved its score. Dropout prevents the model from just memorizing the training data, helping it make better predictions on new data.

What Did Not Help:

- **CNN Model:** The CNN only looked at the very last weather frame and completely ignored the past frames. Because weather prediction relies heavily on seeing how clouds and rain move over time, losing this history limited its accuracy.
- **LSTM Model:** While LSTMs are usually good with time-series data, it struggled here. Flattening the 2D weather images into a simple 1D line of numbers made it very difficult for the LSTM to understand the spatial weather patterns.